CHANDIGARH UNIVERSITY

UIE – University Institute of Engineering

PROJECT REPORT

QUANTUM ROUTE PATHFINDING SIMULATOR

Dynamic Multi-Criteria Pathfinding System

Submitted in partial fulfilment of the requirements for

ADVANCED DATA STRUCTURES (ADS)

Submitted By

Ansh | UID: 25BCS11004

Aryan Vaid | UID: 25BCS10791

Section: 802-A

Submitted to

Md. Tauseef Sir

Project Repository:
https://github.com/25bcs11004-blip/Quantum-Route-Pathfinding-Simulator

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty and the Department of UIE – University Institute of Engineering at Chandigarh University for the academic guidance and opportunity to develop this Advanced Data Structures project.

We thank Md. Tauseef Sir for the guidance, feedback and encouragement that supported the planning and implementation of the simulator. We also acknowledge the authors and maintainers of the C++ language documentation and the foundational algorithmic literature that informed our understanding of graph search, disjoint-set union, segment trees and randomized evaluation.

Finally, we acknowledge the use of the project repository and the inspected source code as the primary implementation reference.

ABSTRACT

The Quantum Route Pathfinding Simulator is a C++20 command-line application that models a changing two-dimensional environment as a weighted grid graph. An agent must move from a source cell to a destination while avoiding blocked cells and edges and accounting for movement distance, traffic congestion and environmental risk. The grid contains 35 columns and 21 rows, giving 735 indexed cells. The simulator supports four-direction and eight-direction movement, with diagonal steps assigned a distance of $\sqrt{2}$ and diagonal corner-cutting prevented.

The implementation compares Dijkstra's algorithm and A* search using a priority queue and parent tracking. A Disjoint Set Union (DSU) structure checks whether the source and destination are connected before a full search is attempted. An iterative segment tree maintains per-cell traffic factors and supports point updates, range-sum queries and range-maximum queries. A command interpreter allows users to change presets, obstacles, traffic, risk, weights, movement mode and simulation behaviour.

A distinctive feature is a classical quantum-inspired candidate-selection routine. It generates up to twelve A* candidate paths, including a baseline path and noisy-traffic variants, removes duplicate paths, evaluates the remaining candidates over 48 Monte Carlo traffic scenarios, and scores each path using mean cost plus one-half of its standard deviation. It then applies Grover-style phase inversion and diffusion arithmetic to candidate amplitudes and selects the candidate with the highest normalized probability. These calculations are performed with ordinary C++ data structures and floating-point arithmetic; no qubits, quantum circuit, quantum hardware or demonstrated quantum speedup are involved.

The inspected source compiled successfully with GCC 14.2.0 using the C++20 build command. Deterministic-seed execution checks covered all four environment presets, Dijkstra/A* cost agreement, blocked-edge connectivity detection, traffic updates, segment-tree queries, parameter changes and live delivery. The report presents measured outputs as observations from those runs and distinguishes them from general performance claims.

Keywords: pathfinding, weighted grid graph, Dijkstra, A, DSU, segment tree, Monte Carlo simulation, quantum-inspired optimization, dynamic replanning.*

TABLE OF CONTENTS

1. Introduction

2. Problem Statement and Objectives

3. Literature Review and Algorithm Background

4. System Requirements and Technology Stack

5. System Architecture and Design

6. Data Structures Used

7. Algorithms and Their Implementation

Complete Source Code (main.cpp)

8. Complete Project Working

9. Complexity Analysis

10. Testing and Results

11. Advantages, Limitations and Applications

12. Conclusion

13. Code Explanation
References

Appendix A — Command Quick Reference

CHAPTER 1 — INTRODUCTION

1.1 Introduction to Pathfinding

Pathfinding is the task of finding a valid route from one location to another. In computer science, locations are represented as vertices (nodes) and legal movements are represented as edges. When every movement has the same cost, the problem may be treated as an unweighted shortest-path problem. When movements have different costs, the graph is weighted and the route must be chosen by accumulating edge costs.

A grid is a convenient model for a map because each cell can represent a small region. A cell may be open, blocked, congested or risky. In this project, the grid is not stored as a full adjacency-list graph. Instead, legal neighboring cells are generated when a search expands a cell. This implicit representation is compact and naturally matches four-direction and eight-direction movement.

1.2 Importance of Route Optimization

A route with the fewest steps is not always the best route. A slightly longer route may avoid heavy traffic or a hazardous area. The simulator therefore treats route selection as a multi-criteria optimization problem. The selected route is influenced by distance, estimated travel time and risk, each controlled by a user-adjustable weight.

1.3 Real-World Applications

- Navigation and logistics: choosing routes that balance distance and congestion.
- Emergency response: avoiding blocked roads and high-risk zones.
- Robotics: moving an agent through a workspace containing obstacles.
- Warehouse automation: planning movements around restricted aisles and busy zones.
- Game AI: finding a route through a changing tile-based world.

1.4 Problems with Conventional Shortest-Path Approaches

A conventional shortest-path routine usually optimizes one fixed edge-cost model. If traffic, risk or obstacles change, the old route can become unsuitable. A purely distance-based route may pass through congested or risky cells. Re-running a full search for every small change can also be expensive. This project addresses these concerns by combining weighted edge costs, a connectivity pre-check, efficient traffic queries and replanning when the world version changes.

1.5 Dynamic Multi-Criteria Routing

The simulator calculates the cost of entering a destination cell j as:

$$\mathbf{StepCost(j, d) = w_dist \times d + w_time \times d \times f_j + w_risk \times risk_j}$$

Here d is 1 for a horizontal or vertical step and √2 for a diagonal step; f_j is the traffic factor for cell j; risk_j is the risk value of the destination cell. The weights express the relative importance of distance, time and risk and must be non-negative and not all zero.

1.6 Quantum-Inspired Optimization

The project uses the phrase quantum-inspired to describe a classical numerical procedure that borrows the form of amplitude initialization, phase inversion and diffusion from Grover-style search. The implementation first evaluates a finite list of candidate paths using randomized traffic scenarios. It then transforms ordinary floating-point amplitudes and derives probabilities from their squares. This is an educational analogy, not a physical quantum computation.

1.7 Proposed Project Overview

The complete application consists of a grid-based World model, DSU connectivity structure, iterative traffic segment tree, Dijkstra/A* search routine, quantum-inspired candidate evaluator, terminal renderer and command interpreter. The main contribution is an integrated ADS demonstration combining graph traversal, priority queues, DSU, segment trees, hashing, vectors and randomized simulation.

CHAPTER 2 — PROBLEM STATEMENT AND OBJECTIVES

2.1 Detailed Problem Statement

Given a rectangular grid of 35 columns and 21 rows, a source cell, a destination cell and a changing environment, find a legal path that minimizes a weighted cost. Some cells may be blocked, some adjacent cell-pairs may have a blocked edge, and open cells may carry different traffic and risk values. The movement policy may permit four or eight directions. A valid solution must detect when no route exists, compare classical search strategies and support re-planning after the environment changes.

2.2 Existing Problems

- Distance-only routing ignores congestion and risk.
- Static path plans can become invalid after an obstacle or traffic change.
- Search should be avoided when the source and destination are already disconnected.
- Repeated traffic updates require efficient aggregate analytics.
- Algorithm behaviour is difficult to understand without visual search traces and route overlays.

2.3 Proposed Solution

The proposed simulator models the map as an implicit weighted graph. Dijkstra and A* are used for route search, while DSU performs a connectivity check. Traffic factors are maintained by an iterative segment tree. A candidate-selection stage uses noisy A* searches and Monte Carlo evaluation to compare multiple routes under uncertainty. The command interpreter allows the user to modify the world and observe replanning.

2.4 Main Objectives

1. Represent a 35 × 21 environment as a row-major grid graph.
2. Implement Dijkstra and A* using a min-priority queue and parent array.
3. Support four-direction and eight-direction movement without diagonal corner-cutting.
4. Calculate composite route costs using distance, traffic and risk.
5. Use DSU to verify connectivity before expensive search.
6. Use a segment tree for traffic point updates and range aggregation.
7. Generate and evaluate multiple candidate paths using randomized traffic scenarios.
8. Demonstrate Grover-style amplitude arithmetic as a classical quantum-inspired selection layer.
9. Support interactive commands, live traffic variation, route replanning and delivery.

2.5 Project Scope

The project is a self-contained command-line simulation. It does not connect to real maps, GPS, live traffic feeds, physical robots or quantum hardware. Its environment is a synthetic grid with randomized presets. The implementation demonstrates data structures and algorithmic techniques rather than providing a production navigation service.

2.6 Expected Outcomes

The expected outcome is a working educational simulator in which students can observe how a weighted graph search reacts to obstacles, congestion, risk and changing priorities. It should make the trade-off between shortest distance and composite cost visible and provide measurable search statistics. A successful run should also demonstrate that A* can reduce the number of expanded nodes while preserving the Dijkstra cost when the heuristic is admissible.

CHAPTER 3 — LITERATURE REVIEW AND ALGORITHM BACKGROUND

3.1 Graph Theory

A graph G=(V,E) consists of a set V of vertices and a set E of edges connecting pairs of vertices. In this simulator, each traversable grid cell is treated as a vertex. A legal move between neighboring cells is an edge. Edge cost is computed when a neighbor is examined, rather than being stored permanently.

3.2 Weighted and Unweighted Graphs

In an unweighted graph, all edges are treated equally and the objective is often to minimize the number of edges. In a weighted graph, each edge has a numerical cost. The project uses a weighted graph because moving into cells with higher traffic or risk should be more expensive.

3.3 Grid-Based Graph Representation

The grid is stored using a single integer index. For column count C=35, cell (x,y) maps to index y×35+x. Conversely, x=index mod 35 and y=floor(index/35). The total number of cells is V=35×21=735. Legal neighbors are generated on demand using fixed offset arrays. The dir8 flag selects four or eight offsets.

3.4 Dijkstra's Algorithm

Dijkstra's algorithm finds minimum-cost paths from a source when edge costs are non-negative. It repeatedly removes the unsettled vertex with the smallest known distance from a min-priority queue, then relaxes its outgoing edges. Relaxation means checking whether reaching a neighbor through the current vertex improves its best-known cost. The source uses non-negative weights and a closed array to prevent re-expansion.

3.5 A* Search Algorithm

A* ranks a search state using f(v)=g(v)+h(v), where g(v) is the cost already paid and h(v) estimates the remaining cost to the destination. The project uses a lower-bound per-unit cost multiplied by a geometric distance. With four-direction movement, the default geometry is Manhattan distance; with eight-direction movement it uses Chebyshev distance. The heur euclid command selects Euclidean distance.

3.6 Heuristics and Admissibility

A heuristic is admissible when it never overestimates the true remaining cost. The source computes a minimum possible unit cost across open cells and scales it by the geometric distance. This provides a conservative lower bound for the implemented movement model. An admissible heuristic helps A* prioritize promising nodes while preserving optimality under the search assumptions.

3.7 Disjoint Set Union

Disjoint Set Union maintains a collection of non-overlapping sets. find(x) returns the representative of the set containing x. unite(a,b) merges two sets. Path compression shortens future find operations, while union by rank attaches the shallower tree below the deeper tree. The amortized cost per operation is nearly constant, commonly written O(α(V)).

3.8 Segment Trees

A segment tree stores aggregate information over intervals. The source uses an iterative array-based tree with two arrays: one for sums and one for maximum values. Traffic values occupy the leaves. A point update changes one leaf and recomputes its ancestors; a range query combines the relevant tree nodes. Both update and query operations take O(log V) time.

3.9 Monte Carlo Simulation

Monte Carlo simulation estimates the behaviour of a system by evaluating it under repeated randomized scenarios. Here, each scenario creates a vector of multiplicative traffic-noise values. Every candidate path is re-evaluated under that scenario. The mean cost estimates typical performance, while standard deviation measures variability.

3.10 Quantum-Inspired Optimization Concepts

Grover-style amplitude amplification is associated with changing the sign of a marked amplitude and reflecting amplitudes about their mean. The source reproduces these arithmetic steps on a classical vector. It does not represent a quantum state on qubits and does not implement quantum gates or measurement. The algorithmic stages are included to teach the mathematical analogy and must not be interpreted as evidence of quantum advantage.

CHAPTER 4 — SYSTEM REQUIREMENTS AND TECHNOLOGY STACK

4.1 Programming Language and Development Environment

Component	Project details
Programming language	C++20
IDE / editor	Visual Studio Code (editor; not a runtime dependency)
Compiler	GCC / G++; verified with GCC 14.2.0
Build system	Direct compiler command; no external build system required
External libraries	None identified; the source uses standard C++ headers
Interface	Interactive command-line terminal with optional ANSI styling

4.2 Standard Library Headers Used

The implementation relies on: <algorithm>, <chrono>, <cmath>, <cstdint>, <functional>, <iomanip>, <iostream>, <limits>, <numeric>, <queue>, <random>, <sstream>, <string>, <thread>, <unordered_set>, <utility> and <vector>. No external library dependency was identified.

4.3 Hardware and Software Requirements

Suggested minimum: dual-core processor, 4 GB RAM, modest free storage and a keyboard-enabled terminal. Compatible with Windows 10/11, Linux or macOS with a C++20 compiler. A terminal that supports ANSI escape sequences is recommended; color off disables colour codes.

4.4 Build and Execution Commands

g++ -std=c++20 -O2 main.cpp -o quantum_route

./quantum_route 2024

The optional numeric argument sets the random seed (default 2024).

4.5 Project Repository

https://github.com/25bcs11004-blip/Quantum-Route-Pathfinding-Simulator — inspected source is the C++20 main.cpp file (~35 KB).

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The command-line user interacts with the Sim controller through textual commands. Sim owns the World, weights, random generator, agent position and current plan. The World stores walls, risk, traffic, cut edges, movement mode, DSU and segment tree. Search routines query the world for neighbors and step costs. Candidate selection receives paths from repeated A* searches and returns the selected path. The renderer displays the grid, search traces, candidates and statistics.

Figure 5.1 Overall System Architecture

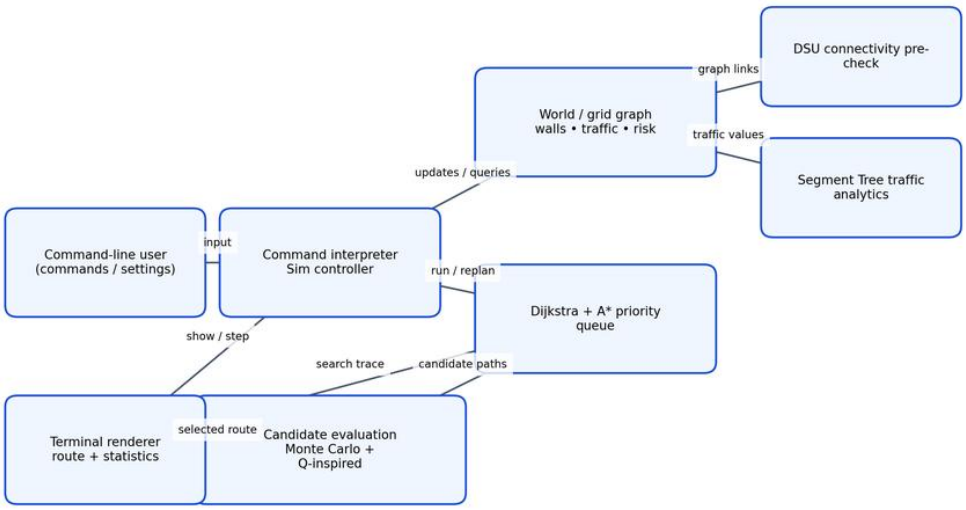


Figure 5.1 Overall System Architecture

5.2 Grid-Based Environment Representation

The grid has 21 rows and 35 columns. Initial source is index 36 (row 1, column 1); destination is index 698 (row 19, column 33). The environment generator creates obstacles according to the selected preset and then explicitly opens the source and goal.

5.3 Graph Representation

The graph is implicit. For a cell i the code calculates x=i%35 and y=i/35, then checks neighboring coordinate offsets. A neighbor is ignored if it lies outside the grid, is a wall, is separated by a blocked edge, or would require diagonal corner-cutting. Four-direction movement uses four offsets; eight-direction uses eight. Diagonal edge distance is √2.

Figure 5.2 Row-Major Grid Representation

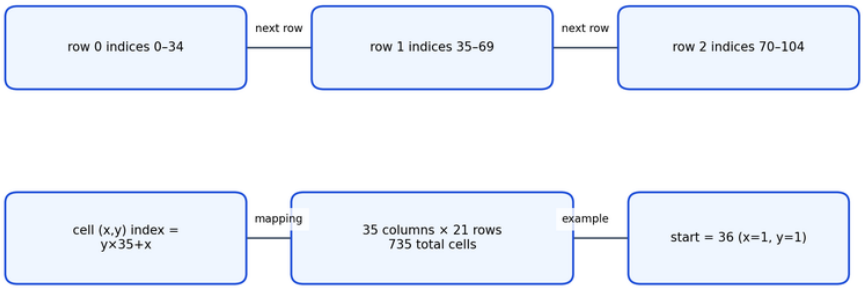


Figure 5.2 Row-Major Grid / Implicit Weighted Graph Representation

5.4 Data Flow

A command enters the parser and is validated. A state-changing command updates the world and increments a version counter. A route request triggers a DSU connectivity check. If connected, the search and optional candidate-evaluation stages execute. The selected path and metrics are sent to the renderer. A change in world version invalidates the existing plan and causes a fresh A* route to be calculated before the next move.

5.5 Major Module Interaction

Logical modules: (1) command interpreter, (2) World model, (3) DSU connectivity, (4) segment-tree traffic analytics, (5) Dijkstra/A* search, (6) quantum-inspired candidate selector, (7) agent movement/delivery, (8) terminal renderer.

CHAPTER 6 — DATA STRUCTURES USED

6.1 Vector

Dynamically sized arrays for walls, risk, distances, parents, closed flags, traffic segment-tree arrays, candidate paths and overlays. Fixed-size grid vectors hold 735 entries. Constant-time indexed access.

6.2 Priority Queue / Min Heap

Returns the element with the smallest priority. Declared with std::greater over pairs of (priority, cell index). Dijkstra inserts g-cost; A* inserts g+h. Push/pop take O(log V).

6.3 Graph / Grid Representation

World represents the graph through grid arrays and a neighbor-generation function. No separate adjacency list. For a bounded-degree grid E is O(V).

6.4 Unordered Set

cutEdges stores blocked edges. Edge key is order-independent (sorted indices combined as a*N+b). Expected O(1) insertion and lookup.

6.5 Disjoint Set Union

Maintains connected components after walls and edge cuts. Path compression and union by rank. Connectivity checked by comparing representatives of agent and goal.

6.6 Segment Tree

Iterative, arrays of length 2N for sums and maxima. Leaves at N..2N−1. Point update and range query O(log V).

6.7 Parent Array and Candidate Storage

Parent array stores previous cell on best path for reconstruction. Candidates store path vectors plus mean, SD, score, amplitude and probability (at most 12 unique paths).

6.8 Data Structure Comparison

Structure	Project use	Typical operation	Time	Space
Vector	Grid/search state	Index access	O(1)	O(V)
Priority queue	Search frontier	Push/pop	O(log V)	O(E)
Implicit grid	Legal neighbors	Generate neighbors	O(1)/cell	O(V)
Unordered set	Blocked edges	Insert / lookup	O(1) exp.	O(B)
DSU	Connectivity	Find / union	O(α(V))	O(V)
Segment tree	Traffic statistics	Update / query	O(log V)	O(V)
Parent array	Path reconstruction	Follow parent	O(L)	O(V)

CHAPTER 7 — ALGORITHMS AND THEIR IMPLEMENTATION

7.1 Dijkstra's Algorithm

Finds the minimum composite-cost route when all step costs are non-negative. Initializes distances to infinity and source to zero. Min-priority queue drives expansion. Relaxes legal neighbors; parent chain reconstructs the route. Complexity O((V+E) log V); on this grid O(V log V).

Flowchart 9.1 Overall Program Execution

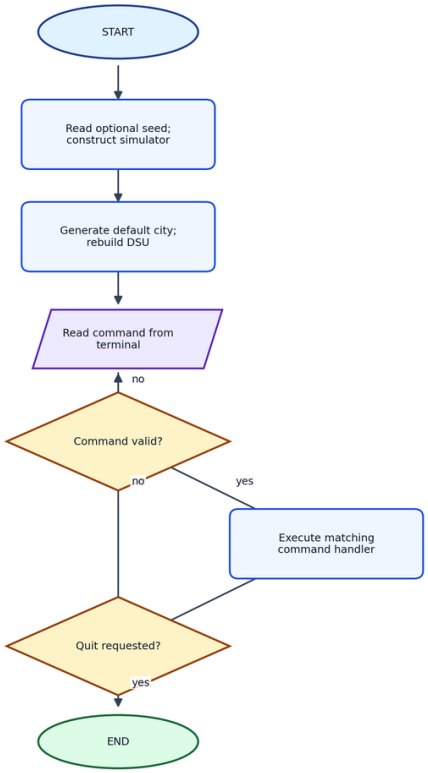


Figure 7.1 Dijkstra Search Flowchart

7.2 A* Search Algorithm

Same relaxation as Dijkstra but prioritizes g+h. Heuristic uses Manhattan (4-dir), Chebyshev (8-dir) or Euclidean. Multiplied by minimum unit cost for admissibility. Profile runs each search 25 times and reports average microseconds.

Flowchart 9.4 DSU Connectivity Verification

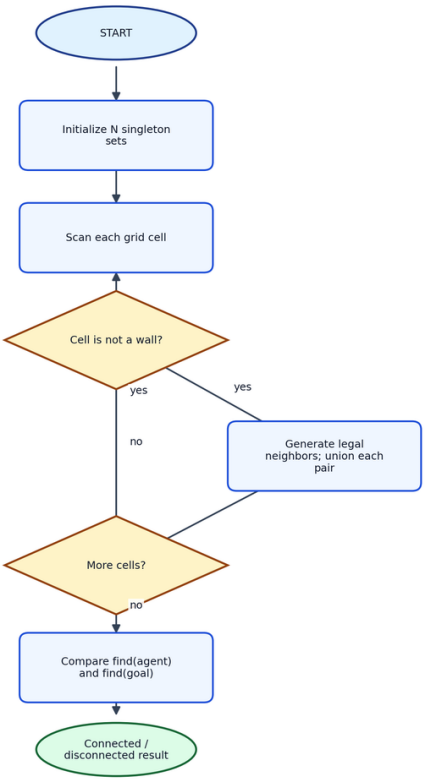


Figure 7.2 A Search Flowchart*

7.3 DSU Operations

find with path compression; unite with union by rank. Rebuild resets and unions legal neighbor pairs for non-wall cells. Before search, compare find(agent) and find(goal); if different, skip search.

7.4 Segment Tree Operations

Initialized with traffic factor 1.0. set changes one leaf and pulls ancestors. Range methods use half-open intervals. query l r validates inclusive range and calls [l,r+1).

7.5 Monte Carlo Candidate Evaluation

48 traffic-noise scenarios with factors exp(sigma × Z). Mean μ = sum/48; σ = sqrt(max(0, sumSquares/48 − μ²)). Score = μ + 0.5σ. Lower scores preferred.

7.6 Quantum-Inspired Selection

Up to 12 A* searches (baseline + noisy). Duplicates removed. Amplitudes start at 1/√M. Marked candidate (min score) has sign flipped; amplitudes reflected about mean for ~(π/4)√M iterations. Normalized squared amplitudes become probabilities; highest probability is selected. Classical floating-point only — no qubits or quantum speedup.

7.7 Dynamic Path Replanning

State-changing commands increment a version counter. Before movement, if plan is stale, replan rebuilds DSU, verifies connectivity and runs A* from current agent. Live mode modifies eight random traffic cells per tick, replans and advances the agent.

COMPLETE SOURCE CODE (main.cpp)

The complete inspected C++20 source file (main.cpp, 610 lines). Build: g++ -std=c++20 -O2 main.cpp -o quantum_route

```
// Quantum Route Lab - Dynamic Multi-Criteria Pathfinding Simulator (C++20) // Build: g++ -std=c++20 -O2
main.cpp -o quantum_route // Run:   ./quantum_route [seed]      (type "help" inside) // Grid is 35x21, cell
index = y*35 + x.  ANSI terminal recommended ("color off" disables).  #include <algorithm> #include <chrono>
#include <cmath> #include <cstdint> #include <functional> #include <iomanip> #include <iostream> #include
<limits> #include <numeric> #include <queue> #include <random> #include <sstream> #include <string> #include
<thread> #include <unordered_map> #include <unordered_set> #include <utility> #include <vector>  namespace
Ansi { inline bool on = true; inline std::string fg(int c) { return on ? "\033[38;5;" + std::to_string(c) +
"m" : ""; } inline std::string bg(int c) { return on ? "\033[48;5;" + std::to_string(c) + "m" : ""; } inline
std::string bold() { return on ? "\033[1m" : ""; } inline std::string rst() { return on ? "\033[0m" : ""; }
inline std::string eol() { return on ? "\033[K" : ""; } inline std::string up(int n) { return on ? "\033[" +
std::to_string(n) + "A\r" : ""; } }  // namespace Ansi  constexpr int C = 35, R = 21, N = C * R; constexpr
double INF = std::numeric_limits<double>::infinity(); constexpr double SQ2 = 1.41421356237309505; constexpr
double PI = 3.14159265358979323846;  // ---------------------------------------------------------------- DSU
class DSU {     std::vector<int> p_, r_;     int comps_ = 0; public:     void reset(int n) { p_.resize(n);
std::iota(p_.begin(), p_.end(), 0); r_.assign(n, 0); comps_ = n; }     int find(int x) {         int r = x;
while (p_[r] != r) r = p_[r];         while (p_[x] != r) { int n = p_[x]; p_[x] = r; x = n; }  // path
compression         return r;     }     void unite(int a, int b) {         a = find(a); b = find(b);
if (a == b) return;         if (r_[a] < r_[b]) std::swap(a, b);                     // union by rank
p_[b] = a;         if (r_[a] == r_[b]) ++r_[a];         --comps_;     }     bool connected(int a, int b) {
return find(a) == find(b); }
    int components() const { return comps_; } };  // ----------------------------------------------------------
-------- Segment tree (traffic factors) class SegTree {     int n_ = 0;     std::vector<double> s_, m_;
void pull(int i) { s_[i] = s_[2 * i] + s_[2 * i + 1]; m_[i] = std::max(m_[2 * i], m_[2 * i + 1]); } public:
void init(int n, double v) {         n_ = n; s_.assign(2 * n, 0.0); m_.assign(2 * n, -INF);         for (int
i = 0; i < n; ++i) s_[n + i] = m_[n + i] = v;         for (int i = n - 1; i > 0; --i) pull(i);     }     void
set(int p, double v) { int i = p + n_; s_[i] = m_[i] = v; for (i >>= 1; i > 0; i >>= 1) pull(i); }  // O(log
K)     double get(int p) const { return s_[p + n_]; }     double sum(int l, int r) const {  // [l, r)
double a = 0;         for (l += n_, r += n_; l < r; l >>= 1, r >>= 1) { if (l & 1) a += s_[l++]; if (r & 1) a
+= s_[--r]; }         return a;     }     double max(int l, int r) const {         double a = -INF;
for (l += n_, r += n_; l < r; l >>= 1, r >>= 1) { if (l & 1) a = std::max(a, m_[l++]); if (r & 1) a =
std::max(a, m_[--r]); }         return a;     } };  // ----------------------------------------------------------
---------- World (dynamic graph) struct Weights { double wd = 1, wt = 1, wr = 1; };
struct Totals { double d = 0, t = 0, r = 0, c = 0; };  struct World {     std::vector<char> wall =
std::vector<char>(N, 0);     std::vector<float> risk = std::vector<float>(N, 0.f);
std::unordered_set<long long> cutEdges;  // blocked edges (u-v)     SegTree traffic;     DSU dsu;     bool
dir8 = false;      World() { traffic.init(N, 1.0); }     static long long ekey(int a, int b) { if (a > b)
std::swap(a, b); return static_cast<long long>(a) * N + b; }     bool edgeCut(int a, int b) const { return
cutEdges.count(ekey(a, b)) > 0; }      template <class F>     void eachNb(int i, F fn) const {         static
const int DX[8] = {1, -1, 0, 0, 1, 1, -1, -1}, DY[8] = {0, 0, 1, -1, 1, -1, 1, -1};         int x = i % C, y
= i / C, cnt = dir8 ? 8 : 4;         for (int k = 0; k < cnt; ++k) {             int nx = x + DX[k], ny = y +
DY[k];             if (nx < 0 || ny < 0 || nx >= C || ny >= R) continue;             int j = ny * C + nx;
if (wall[j] || edgeCut(i, j)) continue;             if (DX[k] && DY[k] && (wall[y * C + nx] || wall[ny * C +
x])) continue;  // no corner cutting             fn(j, (DX[k] && DY[k]) ? SQ2 : 1.0);         }     }
void rebuildDSU() {         dsu.reset(N);         for (int i = 0; i < N; ++i) if (!wall[i]) eachNb(i, [&](int
j, double) { dsu.unite(i, j); });
    }     // cost of stepping INTO cell j:  wd*dist + wt*(dist/speed) + wr*risk,  speed = 1/(traffic*noise)
double stepCost(int j, double d, const Weights& w, const std::vector<double>* nz) const {         double f =
traffic.get(j) * (nz ? (*nz)[j] : 1.0);         return w.wd * d + w.wt * d * f + w.wr * risk[j];     }
Totals stats(const std::vector<int>& p, const Weights& w, const std::vector<double>* nz = nullptr) const {
Totals t;         for (size_t i = 1; i < p.size(); ++i) {             int a = p[i - 1], b = p[i];
double d = (a % C != b % C && a / C != b / C) ? SQ2 : 1.0;             double f = traffic.get(b) * (nz ?
(*nz)[b] : 1.0);             t.d += d; t.t += d * f; t.r += risk[b]; t.c += stepCost(b, d, w, nz);         }
return t;     }     double unitCost(const Weights& w, const std::vector<double>* nz) const {         double m
= INF;         for (int i = 0; i < N; ++i) if (!wall[i]) {             double v = w.wd + w.wt *
traffic.get(i) * (nz ? (*nz)[i] : 1.0) + w.wr * risk[i] / SQ2;             m = std::min(m, v);         }
return std::isfinite(m) ? m : 0.0;     } };  // ----------------------------------------------------------------
--- Search (Dijkstra / A*) struct Result {     bool found = false;     std::vector<int> path;
    size_t popped = 0, expanded = 0;     double micros = 0;     Totals tot;     std::vector<int> ex, ds;  //
expansion index, discovery index (for animation) };  Result search(const World& g, int s, int t, const
Weights& w, bool useH, bool euclid,               const std::vector<double>* nz) {     Result res;
std::vector<double> gs(N, INF);     std::vector<int> par(N, -1);     std::vector<char> closed(N, 0);
res.ex.assign(N, -1); res.ds.assign(N, -1);     double u = useH ? g.unitCost(w, nz) : 0.0;     int tx = t %
C, ty = t / C;     auto h = [&](int v) -> double {         if (!useH) return 0.0;         double dx =
std::abs(v % C - tx), dy = std::abs(v / C - ty);         double d = euclid ? std::hypot(dx, dy) : (g.dir8 ?
std::max(dx, dy) : dx + dy);         return d * u;  // admissible: u is the cheapest possible cost per unit
distance     };     using QI = std::pair<double, int>;     std::priority_queue<QI, std::vector<QI>,
std::greater<QI>> pq;     gs[s] = 0; res.ds[s] = 0; pq.push({h(s), s});     while (!pq.empty()) {         int
v = pq.top().second; pq.pop();         ++res.popped;         if (closed[v]) continue;         closed[v] = 1;
res.ex[v] = static_cast<int>(res.expanded++);         if (v == t) { res.found = true; break; }
        g.eachNb(v, [&](int j, double d) {             if (closed[j]) return;             double ng = gs[v] +
g.stepCost(j, d, w, nz);             if (ng < gs[j]) {                 gs[j] = ng; par[j] = v;
```

```
if (res.ds[j] < 0) res.ds[j] = static_cast<int>(res.expanded);                 pq.push({ng + h(j), j});
}         });     }     if (res.found) {         for (int v = t; v != -1; v = par[v]) res.path.push_back(v);
std::reverse(res.path.begin(), res.path.end());         res.tot = g.stats(res.path, w);     }     return res;
}  // ---------------------------------------------------------------- Quantum-inspired selection struct Cand
{ std::vector<int> path; double det = 0, mean = 0, sd = 0, score = 0, amp = 0, prob = 0; }; struct QReport {
bool ok = false; std::vector<Cand> c; int chosen = -1, iters = 0; size_t popped = 0, expanded = 0; double
micros = 0; Totals tot; };  QReport quantumSelect(const World& g, int s, int t, const Weights& w, bool
euclid, double sigma, std::mt19937& rng) {     QReport q;     auto t0 = std::chrono::steady_clock::now();
std::normal_distribution<double> nd(0, 1);     auto noise = [&](double sg) { std::vector<double> v(N); for
(auto& x : v) x = std::exp(sg * nd(rng)); return v; };     const int NC = 12, NS = 48;     for (int k = 0; k
< NC; ++k) {                        // 1. superposition of candidate trajectories         std::vector<double>
nz;
        if (k) nz = noise(sigma * 1.8);         Result r = search(g, s, t, w, true, euclid, k ? &nz :
nullptr);         q.popped += r.popped; q.expanded += r.expanded;         if (!r.found) { if (!k) return q;
continue; }         bool dup = false;         for (auto& c : q.c) if (c.path == r.path) { dup = true; break;
}         if (!dup) { Cand c; c.path = r.path; c.det = r.tot.c; q.c.push_back(std::move(c)); }     }     int
M = static_cast<int>(q.c.size());     std::vector<double> sum(M, 0), sq(M, 0);     for (int sc = 0; sc < NS;
++sc) {                     // 2. Monte-Carlo amplitude evaluation         auto nz = noise(sigma);
for (int i = 0; i < M; ++i) { double x = g.stats(q.c[i].path, w, &nz).c; sum[i] += x; sq[i] += x * x; }     }
int best = 0;     for (int i = 0; i < M; ++i) {         q.c[i].mean = sum[i] / NS;         q.c[i].sd =
std::sqrt(std::max(0.0, sq[i] / NS - q.c[i].mean * q.c[i].mean));         q.c[i].score = q.c[i].mean + 0.5 *
q.c[i].sd;         if (q.c[i].score < q.c[best].score) best = i;     }     std::vector<double> amp(M, 1.0 /
std::sqrt(static_cast<double>(M)));     q.iters = M > 1 ? std::max(1, static_cast<int>(std::lround(PI / 4 *
std::sqrt(static_cast<double>(M))))) : 0;     for (int k = 0; k < q.iters; ++k) {                    // 3.
Grover-style oracle + diffusion         amp[best] = -amp[best];         double m =
std::accumulate(amp.begin(), amp.end(), 0.0) / M;         for (auto& a : amp) a = 2 * m - a;     }     double
nrm = 0; for (double a : amp) nrm += a * a;     int ch = 0;
    for (int i = 0; i < M; ++i) {         q.c[i].amp = amp[i]; q.c[i].prob = amp[i] * amp[i] / nrm;
if (q.c[i].prob > q.c[ch].prob + 1e-12) ch = i;   // 4. collapse     }     q.chosen = ch; q.ok = true; q.tot
= g.stats(q.c[ch].path, w);     q.micros = std::chrono::duration<double,
std::micro>(std::chrono::steady_clock::now() - t0).count();     return q; }  // ----------------------------
---------------------------------- Simulator struct Overlay {     std::vector<char> expl, front, cand, path;
Overlay() : expl(N, 0), front(N, 0), cand(N, 0), path(N, 0) {} };  class Sim {     World g;     Weights w;
std::mt19937 rng;     int start = 1 * C + 1, goal = (R - 2) * C + (C - 2), agent = start;
std::vector<int> plan;     int ver = 0, planVer = -1;     bool euclid = false, anim = true, useQ = true;
int frameMs = 25;     double sigma = 0.3, travelled = 0;      static void sleepMs(int ms) {
std::this_thread::sleep_for(std::chrono::milliseconds(ms)); }     bool valid(int v) const { return v >= 0 &&
v < N; }     int rnd(int a, int b) { return std::uniform_int_distribution<int>(a, b)(rng); }     double u01()
{ return std::uniform_real_distribution<double>(0, 1)(rng); }
    void touch() { ++ver; }     void blob(int cx, int cy, int r, const std::function<void(int, double)>& fn)
{         for (int y = 0; y < R; ++y) for (int x = 0; x < C; ++x) {             double d = std::hypot(x - cx,
y - cy);             if (d <= r) fn(y * C + x, 1.0 - d / r);         }     }      // ---- presets     void
generate(const std::string& kind) {         g = World(); plan.clear(); planVer = -1; travelled = 0;
start = 1 * C + 1; goal = (R - 2) * C + (C - 2); agent = start;         auto tr = [&](int i, double t) { if
(!g.wall[i]) g.traffic.set(i, 1 + t); };         auto rk = [&](int i, double v) { if (!g.wall[i]) g.risk[i] =
static_cast<float>(std::max<double>(g.risk[i], std::round(v))); };         if (kind == "city") {
for (int y = 0; y < R; ++y) for (int x = 0; x < C; ++x) if (x % 6 && y % 5 && u01() < 0.93) g.wall[y * C + x]
= 1;             for (int k = 0; k < 6; ++k) blob(rnd(2, C - 3), rnd(2, R - 3), rnd(2, 4), [&](int i, double
t) { tr(i, t * 5); });             for (int k = 0; k < 4; ++k) blob(rnd(2, C - 3), rnd(2, R - 3), rnd(2, 3),
[&](int i, double t) { rk(i, 2 + t * 7); });         } else if (kind == "maze") {
std::fill(g.wall.begin(), g.wall.end(), 1);             std::vector<int> st{1 * C + 1}; g.wall[st[0]] = 0;
while (!st.empty()) {                 int v = st.back(), x = v % C, y = v / C;
std::vector<std::pair<int, int>> o;                 for (auto [a, b] : std::vector<std::pair<int, int>>{{2,
0}, {-2, 0}, {0, 2}, {0, -2}})                     if (x + a > 0 && y + b > 0 && x + a < C - 1 && y + b < R -
1 && g.wall[(y + b) * C + x + a]) o.push_back({a, b});                 if (o.empty()) { st.pop_back();
continue; }                 auto [a, b] = o[rnd(0, static_cast<int>(o.size()) - 1)];
g.wall[(y + b / 2) * C + x + a / 2] = 0; g.wall[(y + b) * C + x + a] = 0;                 st.push_back((y +
b) * C + x + a);
            }             for (int k = 0; k < 40; ++k) g.wall[rnd(1, R - 2) * C + rnd(1, C - 2)] = 0;
for (int i = 0; i < N; ++i) if (!g.wall[i] && u01() < 0.08) g.risk[i] = static_cast<float>(rnd(2, 6));
} else if (kind == "field") {             for (int k = 0; k < 9; ++k) blob(rnd(3, C - 4), rnd(3, R - 4),
rnd(1, 3), [&](int i, double) { g.wall[i] = 1; });             for (int k = 0; k < 5; ++k) blob(rnd(3, C -
4), rnd(3, R - 4), rnd(3, 5), [&](int i, double t) { rk(i, 1 + t * 8); });             for (int k = 0; k < 4;
++k) blob(rnd(3, C - 4), rnd(3, R - 4), rnd(3, 5), [&](int i, double t) { tr(i, t * 4); });         } else {
// disaster             for (int k = 0; k < 22; ++k) blob(rnd(1, C - 2), rnd(1, R - 2), rnd(1, 2), [&](int i,
double) { g.wall[i] = 1; });             for (int k = 0; k < 9; ++k) blob(rnd(2, C - 3), rnd(2, R - 3),
rnd(3, 6), [&](int i, double t) { rk(i, 2 + t * 8); });             for (int k = 0; k < 7; ++k) blob(rnd(2, C
- 3), rnd(2, R - 3), rnd(2, 5), [&](int i, double t) { tr(i, t * 6); });         }         g.wall[start] =
g.wall[goal] = 0;         g.rebuildDSU();         for (int guard = 0; !g.dsu.connected(start, goal) && guard
< 2000; ++guard) { g.wall[rnd(0, N - 1)] = 0; g.rebuildDSU(); }         touch();     }      // ---- rendering
std::string cell(int i, const Overlay& ov) const {         using namespace Ansi;         double f =
g.traffic.get(i);         bool plainMode = !on;         if (i == agent) return plainMode ? "@ " : bg(226) +
fg(16) + bold() + "@ " + rst();         if (i == goal) return plainMode ? "G " : bg(46) + fg(16) + bold() +
"G " + rst();         if (i == start) return plainMode ? "S " : bg(255) + fg(16) + "S " + rst();         if
(g.wall[i]) return plainMode ? "##" : bg(52) + fg(196) + bold() + "XX" + rst();         if (ov.path[i])
return plainMode ? "**" : bg(34) + fg(226) + bold() + "<>" + rst();         if (ov.cand[i]) return plainMode
```

```
? "??" : bg(93) + fg(225) + "::" + rst();         std::string ch = g.risk[i] > 0 ? "!" : (f > 1.05 ? "~" : "
");
        if (g.risk[i] > 0 && f > 1.05) ch = "!";         ch += " ";         if (plainMode) return ov.expl[i]
? ". " : (ov.front[i] ? "o " : ch);         int base = 235;         if (f > 1.05) base = f > 4 ? 202 : (f >
2.5 ? 166 : 130);         if (ov.expl[i]) return bg(30) + fg(51) + bold() + ch + rst();         if
(ov.front[i]) return bg(45) + fg(16) + ch + rst();         return bg(base) + fg(g.risk[i] > 0 ? 205 : 255) +
(g.risk[i] > 0 ? bold() : "") + ch + rst();     }     void draw(const Overlay& ov, const std::string& status,
bool overwrite) const {         std::ostringstream o;         if (overwrite) o << Ansi::up(R + 2);         o
<< Ansi::bold() << status << Ansi::rst() << Ansi::eol() << "\n    ";         for (int x = 0; x < C; ++x) o <<
(x % 5 == 0 ? std::to_string(x % 100) + std::string(x % 5 == 0 && x >= 10 ? 0 : 1, ' ') : std::string("
")).substr(0, 2);         o << Ansi::eol() << "\n";         for (int y = 0; y < R; ++y) {             o <<
std::setw(3) << y * C << " ";             for (int x = 0; x < C; ++x) o << cell(y * C + x, ov);             o
<< Ansi::eol() << "\n";         }         std::cout << o.str() << std::flush;     }     void mark(Overlay&
ov, const std::vector<int>& p, bool cand) const { for (int v : p) (cand ? ov.cand : ov.path)[v] = 1; }
Overlay base() const { Overlay ov; if (!plan.empty()) mark(ov, plan, false); return ov; }     void show(const
std::string& title = "") const {         std::ostringstream st;         st << "agent=" << agent << " goal="
<< goal << " w(d,t,r)=" << w.wd << "," << w.wt << "," << w.wr           << " " << (g.dir8 ? "8-dir" : "4-
dir") << " " << (euclid ? "Euclid" : "Manhattan");         if (!title.empty()) st << " | " << title;
draw(base(), st.str(), false);
    }     void legend() const {         using namespace Ansi;         std::cout << "  " << bg(30) << fg(51)
<< " explored " << rst() << " " << bg(45) << fg(16) << " frontier " << rst() << " "                         <<
bg(93) << fg(225) << " quantum cand. " << rst() << " " << bg(34) << fg(226) << " route " << rst() << " "
<< bg(52) << fg(196) << " blocked " << rst() << " " << bg(166) << " traffic " << rst() << " " << fg(205) <<
"! risk" << rst()                   << "\n  index = row*" << C << " + col (row labels on the left show the
first index of each row)\n";     }     void animateSearch(const Result& r, const std::string& title) const {
Overlay ov;         int total = static_cast<int>(r.expanded), frames = std::max(1, std::min(total, 40)),
batch = (total + frames - 1) / frames;         auto fill = [&](int k) {             for (int v = 0; v < N;
++v) {                 ov.expl[v] = (r.ex[v] >= 0 && r.ex[v] < k);                 ov.front[v] = (!ov.expl[v]
&& r.ds[v] >= 0 && r.ds[v] <= k);             }         };         bool drew = false;         if (anim &&
Ansi::on)             for (int k = batch; k < total; k += batch) { fill(k); draw(ov, title + " expanded " +
std::to_string(k) + "/" + std::to_string(total), drew); drew = true; sleepMs(frameMs); }         fill(total);
mark(ov, r.path, false);         draw(ov, title + " expanded " + std::to_string(total) + "/" +
std::to_string(total) + " -> route found", drew);     }     void animateQuantum(const QReport& q) const {
Overlay ov; bool drew = false;         for (size_t i = 0; i < q.c.size(); ++i) {             mark(ov,
q.c[i].path, true);             if (anim && Ansi::on) { draw(ov, "Quantum superposition: candidate " +
std::to_string(i + 1) + "/" + std::to_string(q.c.size()), drew); drew = true; sleepMs(frameMs * 4); }
}
        mark(ov, q.c[q.chosen].path, false);         draw(ov, "Wavefunction collapsed -> candidate #" +
std::to_string(q.chosen), drew);     }     void printQuantum(const QReport& q) const {         std::cout <<
"\nQuantum register: " << q.c.size() << " superposed paths, sigma=" << sigma << ", Grover iterations=" <<
q.iters << "\n";         std::cout << std::left << std::setw(4) << " #" << std::setw(7) << "Hops" <<
std::setw(11) << "DetCost" << std::setw(11) << "E[cost]"                   << std::setw(9) << "StdDev" <<
std::setw(10) << "Score" << std::setw(10) << "Amp" << "Prob\n";         for (size_t i = 0; i < q.c.size();
++i) {             const Cand& c = q.c[i];             std::ostringstream o;             o << " " <<
std::left << std::setw(3) << i << std::setw(7) << c.path.size() - 1 << std::fixed << std::setprecision(2)
<< std::setw(11) << c.det << std::setw(11) << c.mean << std::setw(9) << c.sd << std::setw(10) << c.score
<< std::setprecision(3) << std::setw(10) << c.amp << std::setprecision(1) << c.prob * 100 << "%";
if (static_cast<int>(i) == q.chosen) o << "  <== collapsed";             if (static_cast<int>(i) == q.chosen)
std::cout << Ansi::fg(165) << Ansi::bold() << o.str() << Ansi::rst() << "\n";             else std::cout <<
o.str() << "\n";         }     }     struct Row { std::string name; double us; size_t popped, expanded;
Totals t; };     void table(const std::vector<Row>& rows) const {         auto sep = [] { std::cout << "+" <<
std::string(18, '-') << "+" << std::string(11, '-') << "+" << std::string(8, '-') << "+" << std::string(10,
'-') << "+" << std::string(11, '-') << "+" << std::string(9, '-') << "+" << std::string(9, '-') << "+" <<
std::string(8, '-') << "+\n"; };         sep();         std::cout << "| " << std::left << std::setw(16) <<
"Algorithm" << " | " << std::setw(9) << "Time(us)" << " | " << std::setw(6) << "Popped" << " | " <<
std::setw(8) << "Expanded"                   << " | " << std::setw(9) << "Cost" << " | " << std::setw(7) <<
"Dist" << " | " << std::setw(7) << "Time" << " | " << std::setw(6) << "Risk" << " |\n";         sep();
for (auto& r : rows)             std::cout << "| " << std::left << std::setw(16) << r.name << " | " <<
std::right << std::fixed << std::setprecision(1) << std::setw(9) << r.us << " | " << std::setw(6) << r.popped
<< " | " << std::setw(8) << r.expanded << " | " << std::setprecision(2) << std::setw(9) << r.t.c << " | " <<
std::setw(7) << r.t.d << " | " << std::setw(7) << r.t.t                       << " | " << std::setw(6) <<
r.t.r << " |\n";         sep();
        std::cout << std::left;     }     Result profile(bool useH, int reps = 25) const {         Result r;
auto t0 = std::chrono::steady_clock::now();         for (int i = 0; i < reps; ++i) r = search(g, agent, goal,
w, useH, euclid, nullptr);         r.micros = std::chrono::duration<double,
std::micro>(std::chrono::steady_clock::now() - t0).count() / reps;         return r;     }     bool replan()
{         g.rebuildDSU();         if (!g.dsu.connected(agent, goal)) { plan.clear(); return false; }
Result r = search(g, agent, goal, w, true, euclid, nullptr);         if (!r.found) { plan.clear(); return
false; }         plan = r.path; planVer = ver; return true;     }  public:     explicit Sim(unsigned seed) :
rng(seed) { generate("city"); }      void help() const {         std::cout << "Commands:\n"
<< "  run                       DSU check, Dijkstra vs A* vs Quantum collapse + stats table\n"
<< "  step                      move agent one node along the optimal route\n"                   << "  live
[n]                   n ticks of fluctuating traffic: re-plan + move each tick (default 15)\n"
<< "  block <u> <v>             cut edge between adjacent cells u,v (DSU rebuilt)   | unblock <u> <v>\n"
<< "  wall <idx> | unwall <idx>  block / free a whole cell\n"                   << "  traffic <u> <v>
<factor>   congestion on edge u-v via segment tree (0.1..100)\n"                   << "  traffic <idx>
```

```
<factor>     congestion on one cell\n"                  << "  risk <idx> <value>         hazard level of a
cell (0..20)\n"
                 << "  weights <w1> <w2> <w3>     w_dist, w_time, w_risk\n"                  << "  query
<l> <r>              segment-tree range query on cell indices [l,r]\n"                   << "  preset
city|maze|field|disaster\n"                   << "  src <idx> | dst <idx>      move start / destination\n"
<< "  dir8 on|off | heur manh|euclid | quantum on|off | sigma <0..1>\n"                   << "  anim
on|off|<ms> | color on|off | show | reset | help | quit\n";     }     void cmdShow() const { show();
legend(); }     void cmdRun() {         if (agent == goal) { std::cout << "Agent is already at the goal.\n";
return; }         g.rebuildDSU();         bool conn = g.dsu.connected(agent, goal);         std::cout <<
"[DSU] components=" << g.dsu.components() << " connected(" << agent << "," << goal << ")=" << (conn ? "true"
: "false") << "\n";         if (!conn) { std::cout << Ansi::fg(196) << "No route exists - search skipped." <<
Ansi::rst() << "\n"; plan.clear(); show("NO ROUTE"); return; }         Result D = profile(false), A =
profile(true);         animateSearch(D, "Dijkstra"); std::cout << "\n";         animateSearch(A, "A*      ");
std::cout << "\n";         std::vector<Row> rows{{"Dijkstra", D.micros, D.popped, D.expanded, D.tot}, {euclid
? "A* (Euclid)" : "A* (Manhattan)", A.micros, A.popped, A.expanded, A.tot}};         QReport q;         if
(useQ) { q = quantumSelect(g, agent, goal, w, euclid, sigma, rng); if (q.ok) rows.push_back({"Quantum
collapse", q.micros, q.popped, q.expanded, q.tot}); }         table(rows);         std::cout << "A* vs
Dijkstra cost match: " << (std::abs(A.tot.c - D.tot.c) < 1e-6 ? "yes" : "NO") << " | A* expanded " <<
std::fixed << std::setprecision(1)                   << 100.0 * (1.0 - static_cast<double>(A.expanded) /
static_cast<double>(D.expanded)) << "% fewer nodes\n";         if (q.ok) {             printQuantum(q);
std::cout << "\n"; animateQuantum(q);             plan = q.c[q.chosen].path;         } else { plan = A.path;
show("A* route"); }         planVer = ver;         legend();     }
    void cmdStep(bool quiet = false) {         if (agent == goal) { std::cout << "Already delivered.\n";
return; }         if (plan.size() < 2 || planVer != ver || plan[0] != agent) {             if (!replan()) {
std::cout << Ansi::fg(196) << "Goal unreachable from " << agent << " (DSU). Agent holds." << Ansi::rst() <<
"\n"; return; }             std::cout << "[replan] graph changed -> new A* route, cost " << std::fixed <<
std::setprecision(2) << g.stats(plan, w).c << "\n";         }         int nxt = plan[1];         travelled +=
g.stats({agent, nxt}, w).c;         std::cout << "Agent " << agent << " -> " << nxt << "\n";         agent =
nxt; plan.erase(plan.begin());         if (agent == goal) { std::cout << Ansi::fg(46) << Ansi::bold() <<
"DELIVERED!" << Ansi::rst() << " travelled composite cost = " << std::fixed << std::setprecision(2) <<
travelled << "\n"; plan.clear(); }         if (!quiet) show();     }     void cmdLive(int n) {         if
(plan.empty() && !replan()) { std::cout << "No route.\n"; return; }         for (int k = 0; k < n && agent !=
goal; ++k) {             for (int j = 0; j < 8; ++j) {                 int i = rnd(0, N - 1);
if (g.wall[i] || i == agent) continue;                 g.traffic.set(i, std::clamp(g.traffic.get(i) + (u01()
< 0.5 ? -1.2 : 1.6), 1.0, 8.0));             }             touch();             if (!replan()) { std::cout <<
"Route lost.\n"; return; }             int nxt = plan[1]; travelled += g.stats({agent, nxt}, w).c; agent =
nxt; plan.erase(plan.begin()); planVer = ver;             if (agent == goal) plan.clear();
std::ostringstream st; st << "LIVE tick " << k + 1 << "/" << n << " agent=" << agent;
draw(base(), st.str(), k > 0);             sleepMs(std::max(frameMs * 4, 80));         }         if (agent ==
goal) std::cout << "DELIVERED under live traffic. Composite cost " << std::fixed << std::setprecision(2) <<
travelled << "\n";
    }     void cmdBlock(int u, int v, bool b) {         if (!valid(u) || !valid(v)) { std::cout << "Index out
of range 0.." << N - 1 << "\n"; return; }         int dx = std::abs(u % C - v % C), dy = std::abs(u / C - v /
C);         if (std::max(dx, dy) != 1 || (!g.dir8 && dx + dy != 1)) { std::cout << "Cells are not
adjacent.\n"; return; }         if (b) g.cutEdges.insert(World::ekey(u, v)); else
g.cutEdges.erase(World::ekey(u, v));         g.rebuildDSU(); touch();         std::cout << "Edge " << u << "-
" << v << (b ? " BLOCKED" : " RESTORED") << ". DSU components=" << g.dsu.components() << " connected(" <<
agent << "," << goal << ")=" << (g.dsu.connected(agent, goal) ? "true" : "false") << "\n";         show();
}     void cmdWall(int i, bool b) {         if (!valid(i) || i == agent || i == goal) { std::cout << "Invalid
cell.\n"; return; }         g.wall[i] = b; g.rebuildDSU(); touch();         std::cout << "Cell " << i << (b ?
" blocked" : " freed") << ". DSU components=" << g.dsu.components() << "\n"; show();     }     void
cmdTraffic(const std::vector<double>& a) {         auto setf = [&](int i, double f) { if (!valid(i)) return
false; g.traffic.set(i, f); return true; };         if (a.size() == 3) {             int u =
static_cast<int>(a[0]), v = static_cast<int>(a[1]); double f = a[2];             if (!valid(u) || !valid(v)
|| f < 0.1 || f > 100) { std::cout << "Bad arguments (factor 0.1..100).\n"; return; }             if
(std::max(std::abs(u % C - v % C), std::abs(u / C - v / C)) != 1) { std::cout << "Cells are not adjacent.\n";
return; }             setf(u, f); setf(v, f);             std::cout << "Edge " << u << "-" << v << " traffic
factor -> " << f << " (segment-tree point updates, O(log " << N << "))\n";         } else if (a.size() == 2)
{             int i = static_cast<int>(a[0]); double f = a[1];             if (!valid(i) || f < 0.1 || f >
100) { std::cout << "Bad arguments.\n"; return; }             setf(i, f); std::cout << "Cell " << i << "
traffic factor -> " << f << "\n";         } else { std::cout << "Usage: traffic <u> <v> <factor> | traffic
<idx> <factor>\n"; return; }         touch();         std::cout << std::fixed << std::setprecision(2) <<
"Network delay index (segment-tree sum - K): " << g.traffic.sum(0, N) - N << "\n";
        show();     }     void cmdRisk(int i, double v) { if (!valid(i) || v < 0 || v > 20) { std::cout <<
"Bad arguments.\n"; return; } g.risk[i] = static_cast<float>(v); touch(); show(); }     void
cmdWeights(double a, double b, double c) {         if (a < 0 || b < 0 || c < 0 || a + b + c <= 0) { std::cout
<< "Weights must be >= 0 and not all zero.\n"; return; }         w = {a, b, c}; touch(); std::cout <<
"Priorities: w_dist=" << a << " w_time=" << b << " w_risk=" << c << " (use 'run' to re-plan)\n";     }
void cmdQuery(int l, int r) {         if (l < 0 || r >= N || l > r) { std::cout << "Need 0 <= l <= r < " << N
<< "\n"; return; }         double s = g.traffic.sum(l, r + 1); int len = r - l + 1;         std::cout <<
std::fixed << std::setprecision(3) << "Cells [" << l << "," << r << "]: total delay=" << s - len << " avg
factor=" << s / len << " max factor=" << g.traffic.max(l, r + 1) << "\n";     }     void cmdPreset(const
std::string& k) {         if (k != "city" && k != "maze" && k != "field" && k != "disaster") { std::cout <<
"Use: preset city|maze|field|disaster\n"; return; }         generate(k); std::cout << "Generated '" << k <<
"'. DSU connected: " << (g.dsu.connected(agent, goal) ? "yes" : "no") << "\n"; show(); legend();     }
void cmdSrc(int v) { if (!valid(v) || v == goal || g.wall[v]) { std::cout << "Invalid.\n"; return; } start =
agent = v; plan.clear(); travelled = 0; touch(); show(); }     void cmdDst(int v) { if (!valid(v) || v ==
```

```
agent || g.wall[v]) { std::cout << "Invalid.\n"; return; } goal = v; plan.clear(); touch(); show(); }
void cmdReset() { agent = start; plan.clear(); travelled = 0; touch(); show("agent reset"); }      void
loop() {         std::cout << Ansi::fg(51) << Ansi::bold() << "=== QUANTUM ROUTE LAB - dynamic multi-criteria
pathfinding ===" << Ansi::rst() << "\nType 'help' for commands.\n";         cmdShow();         std::string
line;         while (true) {             std::cout << Ansi::fg(51) << "quantum-route> " << Ansi::rst() <<
std::flush;             if (!std::getline(std::cin, line)) break;             std::istringstream is(line);
std::string cmd;             if (!(is >> cmd)) continue;
            std::transform(cmd.begin(), cmd.end(), cmd.begin(), [](unsigned char c) { return
static_cast<char>(std::tolower(c)); });             std::vector<std::string> a; std::string t;
while (is >> t) a.push_back(t);             auto num = [&](size_t i, double& out) { if (i >= a.size()) return
false; try { size_t p; out = std::stod(a[i], &p); return p == a[i].size(); } catch (...) { return false; } };
double x, y, z;             if (cmd == "quit" || cmd == "exit" || cmd == "q") break;             else if (cmd
== "help" || cmd == "?") help();             else if (cmd == "show") cmdShow();             else if (cmd ==
"run") cmdRun();             else if (cmd == "step") cmdStep();             else if (cmd == "reset")
cmdReset();             else if (cmd == "live") { double n = 15; num(0, n);
cmdLive(static_cast<int>(std::clamp(n, 1.0, 500.0))); }             else if ((cmd == "block" || cmd ==
"unblock") && num(0, x) && num(1, y)) cmdBlock(static_cast<int>(x), static_cast<int>(y), cmd == "block");
else if ((cmd == "wall" || cmd == "unwall") && num(0, x)) cmdWall(static_cast<int>(x), cmd == "wall");
else if (cmd == "traffic") { std::vector<double> v; double d; for (size_t i = 0; num(i, d); ++i)
v.push_back(d); cmdTraffic(v); }             else if (cmd == "risk" && num(0, x) && num(1, y))
cmdRisk(static_cast<int>(x), y);             else if (cmd == "weights" && num(0, x) && num(1, y) && num(2,
z)) cmdWeights(x, y, z);             else if (cmd == "query" && num(0, x) && num(1, y))
cmdQuery(static_cast<int>(x), static_cast<int>(y));             else if (cmd == "preset" && !a.empty())
cmdPreset(a[0]);             else if (cmd == "src" && num(0, x)) cmdSrc(static_cast<int>(x));
else if (cmd == "dst" && num(0, x)) cmdDst(static_cast<int>(x));             else if (cmd == "dir8" &&
!a.empty()) { g.dir8 = (a[0] == "on"); g.rebuildDSU(); touch(); std::cout << (g.dir8 ? "8" : "4") << "-
direction movement.\n"; }             else if (cmd == "heur" && !a.empty()) { euclid = (a[0][0] == 'e');
touch(); std::cout << "Heuristic: " << (euclid ? "Euclidean" : "Manhattan") << "\n"; }             else if
(cmd == "quantum" && !a.empty()) { useQ = (a[0] == "on"); std::cout << "Quantum filter " << (useQ ? "on" :
"off") << "\n"; }             else if (cmd == "sigma" && num(0, x) && x >= 0 && x <= 1) { sigma = x;
std::cout << "sigma = " << sigma << "\n"; }             else if (cmd == "anim" && !a.empty()) { if (a[0] ==
"on") anim = true; else if (a[0] == "off") anim = false; else if (num(0, x)) frameMs =
static_cast<int>(std::clamp(x, 0.0, 2000.0)); std::cout << "anim " << (anim ? "on" : "off") << ", " <<
frameMs << " ms\n"; }             else if (cmd == "color" && !a.empty()) { Ansi::on = (a[0] == "on");
std::cout << "color " << (Ansi::on ? "on" : "off") << "\n"; }             else std::cout << "Unknown or
malformed command '" << cmd << "'. Type 'help'.\n";         }         std::cout << "Goodbye.\n";
    } };  int main(int argc, char** argv) {     unsigned seed = 2024;     if (argc > 1) { try { seed =
static_cast<unsigned>(std::stoul(argv[1])); } catch (...) {} }     Sim sim(seed);     sim.loop();     return
0; }
```

The following is the complete inspected C++20 source file (main.cpp, 610 lines). It is reproduced here for academic evaluation. Build command: g++ -std=c++20 -O2 main.cpp -o quantum_route

CHAPTER 8 — COMPLETE PROJECT WORKING

8.1 Program Initialization

main starts with seed 2024 (or user argument). mt19937 drives generation. Sim constructor generates the default city preset.

8.2 Environment Preset Selection

Preset	Generation method	Environment effect
City	Random obstacles with road corridors; traffic & risk blobs	Urban-like blocks, roads, congestion
Maze	DFS carving; 40 random openings; some risk	Narrow corridors and dead ends
Field	Nine obstacle, five risk, four traffic blobs	Open terrain with localized obstacles
Disaster	22 obstacle, 9 risk, 7 traffic blobs	Dense disruption and hazards

Flowchart 9.3 Environment Generation

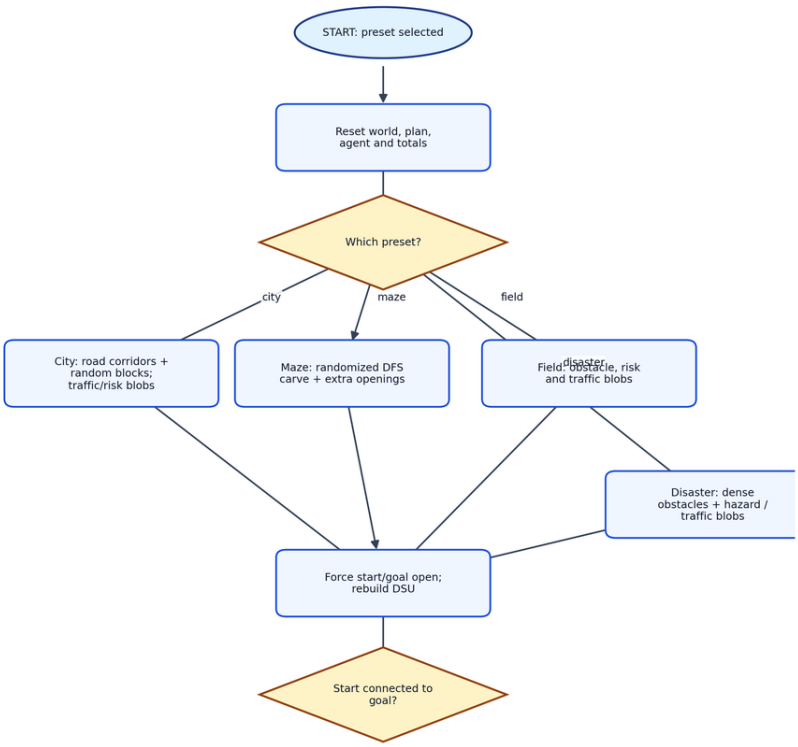


Figure 8.1 Environment Generation Flowchart

8.3 Connectivity, Search and Performance

run rebuilds DSU, checks connectivity, profiles Dijkstra and A* 25 times each, and optionally runs quantum selection. Dijkstra and A* should report equal optimal composite cost under admissible heuristic.

8.4 Agent Movement and Live Simulation

step moves one cell; on goal prints DELIVERED!. live [n] runs up to n ticks of traffic change + replan + move (default 15, max 500).

8.5 Selected Command Reference

Command	Syntax	Purpose

run	run	DSU check; Dijkstra/A*; optional quantum
step / live	step | live [n]	Move one step or multi-tick live sim
wall / traffic	wall <idx> | traffic <idx> <f>	Block cell or set traffic factor
weights / query	weights <wd> <wt> <wr> | query <l> <r>	Set weights or range traffic query
preset / dir8	preset city|maze|... | dir8 on|off	Change environment or movement mode

CHAPTER 9 — COMPLEXITY ANALYSIS

V = 735 cells; E = O(V); M ≤ 12 candidates; S = 48 Monte Carlo scenarios.

Operation	Time complexity	Notes
DSU rebuild	$O((V+E)\alpha(V))$	Reset + union legal pairs
Segment tree update/query	$O(\log V)$	Point or range
Dijkstra / A*	$O((V+E) \log V)$	Binary heap; E=O(V)
Candidate generation	$O(12 \cdot (V+E) \log V)$	At most 12 A* searches
Monte Carlo evaluation	$O(S \cdot M \cdot L)$	48 scenarios × path length
Amplitude stage	$O(M\sqrt{M})$	$\sim(\pi/4)\sqrt{M}$ iterations

9.1 Why the Optimized Structures Matter

Priority queue avoids scanning every unsettled cell. DSU provides a cheap connectivity check. Segment tree makes traffic updates and interval summaries logarithmic. Implicit grid avoids a separate adjacency list.

9.2 Practical Limits

Fixed-size grid keeps memory predictable but does not demonstrate scalability to large networks. Candidate work grows with path count and length; Monte Carlo multiplies by 48 scenarios.

CHAPTER 10 — TESTING AND RESULTS

10.1 Test Environment

Compiled with GCC 14.2.0 (g++ -std=c++20 -O2). Seed 2024. color off and anim off used for clean captures.

10.2 Preset Performance Observations

Preset	Dijk. µs	A* µs	Quantum µs	Cost (D/A*)	A* fewer
City	7.9	5.9	1386.1	110.00	44.4%
Maze	6.1	7.5	1447.4	231.00	1.3%
Field	17.7	7.2	1763.4	100.00	74.8%
Disaster	16.0	12.3	1500.8	167.27	25.6%

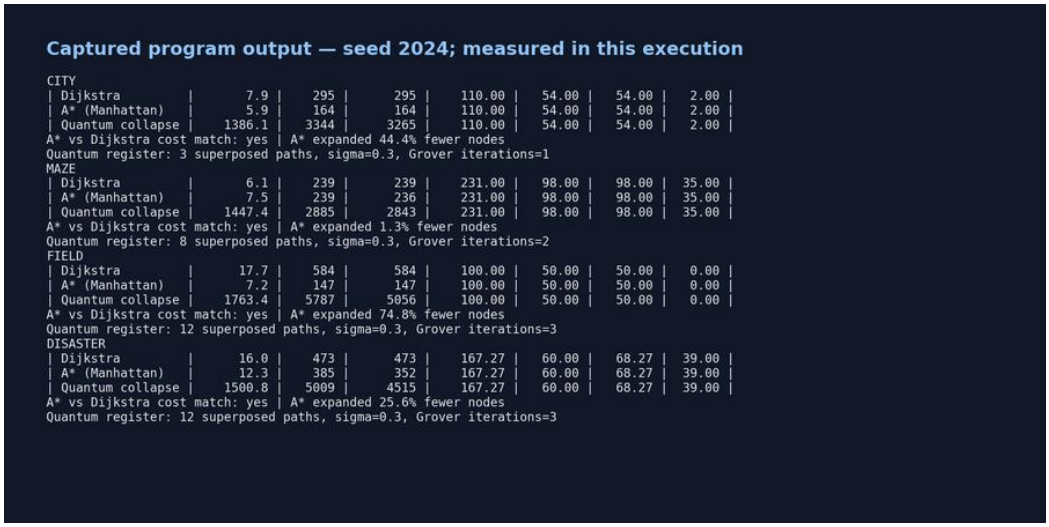


Figure 10.1 Captured performance output (seed 2024)

10.3 Grid and Search Visualizations

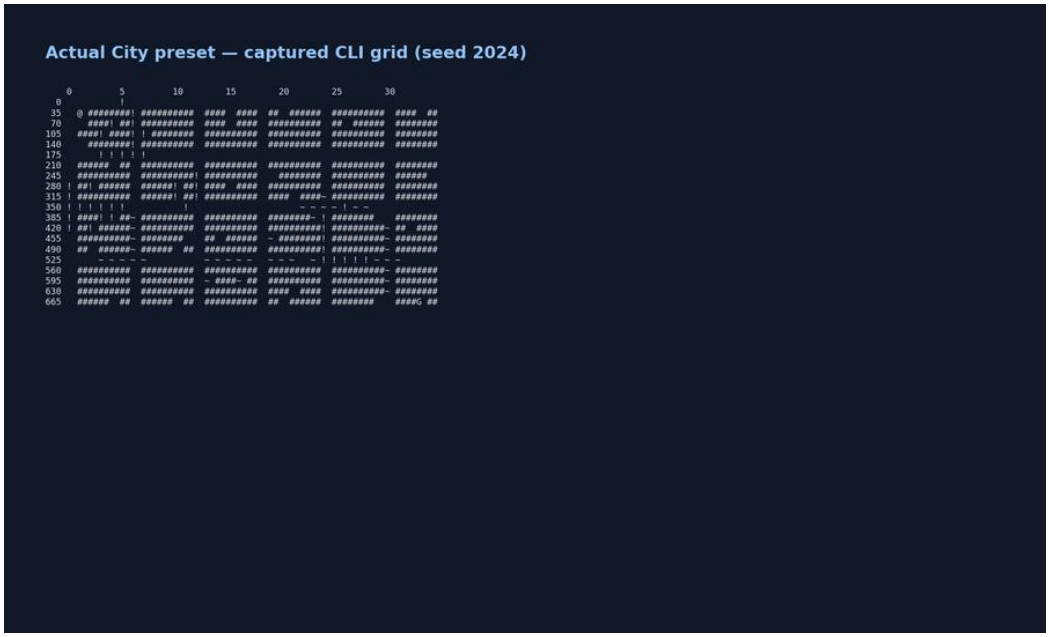


Figure 10.2 Actual City preset grid



Figure 10.3 Actual Dijkstra search visualization — City preset

10.4 Live Simulation and Command Tests



Figure 10.4 Actual live traffic simulation output

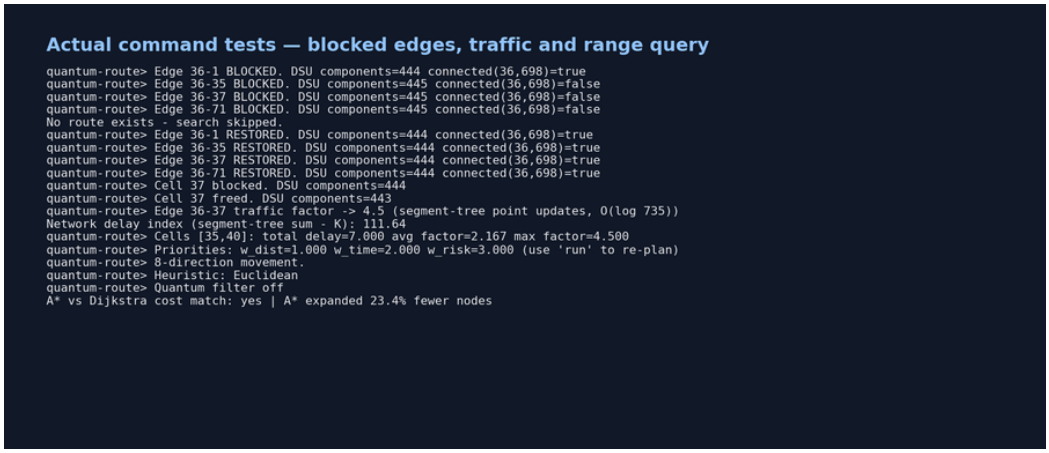


Figure 10.5 Actual obstacle, traffic and Segment Tree query output

10.5 Functional Test Summary

- All four presets generated; Dijkstra/A* costs matched (City 110, Maze 231, Field 100, Disaster 167.27).
- Blocked-route detection via DSU worked; search skipped when disconnected.
- wall/unwall, traffic, risk, weights, dir8, quantum, live delivery and segment-tree query all passed.

10.6 Interpretation

Results confirm correct behaviour on the fixed grid. Quantum-inspired stage is slower (~1.4–1.8 ms) because it runs multiple searches and 48 Monte Carlo scenarios; its purpose is robustness comparison, not speedup. Timings are machine- and seed-dependent.

CHAPTER 11 — ADVANTAGES, LIMITATIONS AND APPLICATIONS

11.1 Advantages

- Multi-criteria cost (distance, congestion, risk).
- Dijkstra vs A* comparison on the same world.
- DSU connectivity pre-check.
- Segment tree for O(log V) traffic updates/queries.
- Dynamic replanning on world changes.
- Standard C++ only; no external dependencies.

11.2 Limitations

- Fixed 35×21 grid.
- CLI interface only.
- Simulated traffic/risk, not real feeds.
- Classical amplitude arithmetic; no quantum speedup.
- DSU rebuild required after topology changes.

11.3 Applications

Educational pathfinding, grid robotics, game AI, warehouse routing, emergency modelling. Synthetic simulator only — not for safety-critical use.

11.4 Future Improvements

- GUI with zoom and live metrics.
- Configurable / importable maps.
- Additional algorithms (D* Lite, multi-objective).
- Multi-seed automated benchmarks.

11.5 Summary Table

Aspect	Advantage	Limitation
Routing cost	Distance, traffic, risk	User-chosen weights
Search	Dijkstra & A* compared	Worst-case similar
Connectivity	Fast DSU check	Rebuild on topology change
Quantum-inspired	Amplitude arithmetic demo	Classical only

CHAPTER 12 — CONCLUSION

The Quantum Route Pathfinding Simulator demonstrates how core Advanced Data Structures and graph algorithms can be combined to solve a dynamic routing problem. A 35×21 grid is modelled as an implicit weighted graph in which obstacles, blocked edges, traffic and risk influence route feasibility and cost. The agent can move using four- or eight-direction movement, and the route can be recalculated when the world changes.

Dijkstra provides a baseline minimum-cost search; A* uses a lower-bound heuristic. DSU verifies connectivity before expensive search. An iterative segment tree supports traffic updates and interval analytics. Vectors, priority queues, parent arrays and an unordered set support core operations.

The quantum-inspired stage generates noisy A* candidates, evaluates them over 48 Monte Carlo scenarios, scores by mean-plus-standard-deviation, and applies Grover-style amplitude arithmetic. The implementation is explicitly classical and should be presented as a mathematical analogy, not actual quantum computing.

Execution checks with GCC 14.2.0 confirmed compilation, all four presets, cost agreement, disconnected-route detection, traffic updates, segment-tree queries, parameter changes and live delivery. Overall, the project is a practical ADS learning environment that connects data structures to a visible, interactive problem while making its technical limitations clear.

REFERENCES

1. Project repository: 25bcs11004-blip, “Quantum-Route-Pathfinding-Simulator,” GitHub. https://github.com/25bcs11004-blip/Quantum-Route-Pathfinding-Simulator (accessed 12 November 2026).

2. Inspected project source: main.cpp, C++20, approximately 35 KB.

3. cppreference.com, C++ standard library reference pages. https://en.cppreference.com/

4. Thomas H. Cormen et al., Introduction to Algorithms, 4th ed., MIT Press, 2022.

5. Peter E. Hart, Nils J. Nilsson and Bertram Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Transactions on Systems Science and Cybernetics, 1968.

Note: Algorithm descriptions are based on the inspected implementation and standard definitions. Measured results are from the inspected source and execution checks.

APPENDIX — COMMAND QUICK REFERENCE

Command	Syntax
Run comparison	run
Move one step	step
Live simulation	live [n]
Block / restore edge	block <u> <v> | unblock <u> <v>
Block / free cell	wall <idx> | unwall <idx>
Set traffic	traffic <idx> <factor> | traffic <u> <v> <factor>
Set risk	risk <idx> <value>
Set weights	weights <wd> <wt> <wr>
Query traffic interval	query <l> <r>
Select preset	preset city|maze|field|disaster
Change endpoints	src <idx> | dst <idx>
Movement / heuristic	dir8 on|off | heur manh|euclid
Candidate / noise	quantum on|off | sigma <0..1>
Display / help / exit	show | reset | help | quit

Commands are lowercased; arguments split by whitespace. Malformed commands produce a help hint. src and dst require valid open cells.

CHAPTER 13 — CODE EXPLANATION

13.1 Overview of the Source File

The complete implementation resides in a single C++20 translation unit (main.cpp, approximately 35 KB / 610 lines). It uses only the C++ standard library. The major components are: ANSI colour helpers, DSU, iterative Segment Tree, World (grid + neighbours + costs), shared Dijkstra/A* search, quantum-inspired candidate selector, Overlay renderer, and the Sim controller with the interactive command loop.

13.2 Grid Constants and Indexing

constexpr int C = 35, R = 21, N = C * R; // 735 cells. Cell (x,y) maps to index y*C + x. This row-major layout makes 1-D vectors natural for walls, risk, distance labels and parents.

13.3 Disjoint Set Union (DSU)

DSU maintains connected components of open cells. Path compression and union-by-rank keep operations nearly O(α(N)). reset creates N singletons; rebuildDSU unions every legal neighbouring pair of non-wall cells. Before any search the program compares find(agent) and find(goal); if they differ the search is skipped.

```
class DSU {
    std::vector<int> p_, r_; int comps_ = 0;
public:
    void reset(int n) { p_.resize(n); std::iota(p_.begin(), p_.end(), 0);
                        r_.assign(n, 0); comps_ = n; }
    int find(int x) {
        int root = x;
        while (p_[root] != root) root = p_[root];
        while (p_[x] != root) { int n = p_[x]; p_[x] = root; x = n; }
        return root;
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (r_[a] < r_[b]) std::swap(a, b);
        p_[b] = a; if (r_[a] == r_[b]) ++r_[a]; --comps_;
    }
    bool connected(int a, int b) { return find(a) == find(b); }
};
```

13.4 Iterative Segment Tree

Two parallel arrays store range sums and range maxima over traffic factors. Leaves occupy indices N … 2N−1. A point update writes a leaf and pulls ancestors; range queries decompose the interval in O(log N).

```
class SegTree {
    int n_ = 0; std::vector<double> s_, m_;
    void pull(int i) { s_[i] = s_[2*i] + s_[2*i+1];
                       m_[i] = std::max(m_[2*i], m_[2*i+1]); }
public:
    void init(int n, double v) { n_ = n; s_.assign(2*n, 0); m_.assign(2*n, -INF);
        for (int i = 0; i < n; ++i) s_[n+i] = m_[n+i] = v;
        for (int i = n-1; i > 0; --i) pull(i); }
    void set(int p, double v) {
        int i = p + n_; s_[i] = m_[i] = v;
        for (i >>= 1; i > 0; i >>= 1) pull(i);
    }
    double sum(int l, int r) const { /* half-open [l,r) O(log n) */ }
    double max(int l, int r) const { /* analogous */ }
};
```

13.5 Step Cost and Neighbour Generation

stepCost(j,d) = wd·d + wt·d·traffic[j]·noise + wr·risk[j]. Diagonal moves use d = √2. eachNb generates up to 8 neighbours, rejecting walls, cut edges and diagonal corner-cutting (when either orthogonal side cell is blocked).

```
double stepCost(int j, double d, const Weights& w,
                const std::vector<double>* nz) const {
    double f = traffic.get(j) * (nz ? (*nz)[j] : 1.0);
    return w.wd * d + w.wt * d * f + w.wr * risk[j];
}
// Diagonal corner-cut prevention:
if (DX[k] && DY[k] &&
    (wall[y*C + nx] || wall[ny*C + x])) continue;
```

13.6 Shared Dijkstra / A* Search

A single search routine implements both algorithms. When useH is false the priority is pure g-cost (Dijkstra); when true the priority is g+h (A*). The heuristic multiplies a geometric distance (Manhattan / Chebyshev / Euclidean) by the minimum observed unit cost, guaranteeing admissibility under the model's non-negative weights.

```
Result search(const World& g, int s, int t, const Weights& w,
              bool useH, bool euclid, const std::vector<double>* nz) {
    // gs = g-scores, pq = min-priority queue of (f, cell)
    // if useH: f = g + h(v); else f = g
    // relax neighbours with stepCost; reconstruct path via parent[]
}
```

13.7 Quantum-Inspired Candidate Selection

Up to 12 A* paths are generated (one clean baseline + noisy traffic variants). Duplicates are removed. Each remaining candidate is scored under 48 Monte Carlo traffic scenarios: score = mean + 0.5·σ. Classical Grover-style iterations then invert the phase of the best-scoring amplitude and reflect all amplitudes about their mean. Squared normalised amplitudes become selection probabilities. No qubits or quantum hardware are used.

```
// Amplitude init: a_i = 1/√M
// For each Grover iteration:
amp[best] = -amp[best];                 // phase inversion
double m = mean(amp);
for (auto& a : amp) a = 2*m - a;        // diffusion
// p_i = a_i² / Σ a_j² ; choose max p_i
```

13.8 Simulator Loop and Commands

Sim owns World, weights, RNG, agent position and the current plan. Every mutating command increments a version counter; step/live replan when the plan is stale. The command interpreter lower-cases the first token and dispatches to handlers (run, step, live, wall, traffic, weights, query, preset, dir8, quantum, etc.).

13.9 Build and Run

g++ -std=c++20 -O2 main.cpp -o quantum_route

./quantum_route 2024

Repository: https://github.com/25bcs11004-blip/Quantum-Route-Pathfinding-Simulator